Elementary data structures solutions

10-E01 Circular-queue states

Derivation

With size=3 starting at head=3, live indices are 3,4,0. Enqueue X writes at tail=1, changes tail to (1+1) mod 5=2, and makes size=4. Dequeue changes head to (3+1) mod 5=4 and size to 3. Enqueue Y writes at index 2, giving tail=3 and size=4.

Result and check

Finally head=4, tail=3, size=4, so the queue is not full. The numerical head/tail order may cross, but size unambiguously defines the live segment.

10-E02 Sentinel deletion order

Derivation

Save prev=A.prev=N and next=A.next=B. Write prev.next=next, then next.prev=prev. Both writes use saved or directly available neighbors and do not depend on rereading A.next afterward.

Result and check

The result has N.next=B, B.prev=N, B.next=N, and N.prev=B. One data node B remains; the list is not empty. If A belongs to another list, these writes may corrupt that list, so membership is a precondition.

10-E03 Left-child/right-sibling

Derivation

left[r]=a, sib[a]=b, sib[b]=c; left[a]=d, sib[d]=e; left[d]=f. All remaining fields without children or siblings are NIL. Start at x=left[a] and follow sib[x] until NIL to enumerate a's children.

Result and check

The children are d,e, a total of 2. Sibling links relate nodes with the same parent; e's next-sibling does not describe a different parent level.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elementary data structures solutions (continued)

10-E04 Unlink before release

Derivation

Allocation returns 1 and leaves free 3→4. Initialize key, prev, and next, then obtain active 0→1→2. Set next[1]=NIL and update the active tail to 1 to unlink 2. Only then write next[2]=3 and free=2. Finally active is 0→1 and free is 2→3→4. Their sets are disjoint and still cover 0..4.

Result and check

If next[1]=2 remains when next[2] is changed to free, active traversal enters the free chain 3→4 through 2. Double release duplicates a free slot or creates a cycle. Release changes slot ownership and the meaning of next; it does not mean clearing the contents.

10-E05 A right sibling is not a child

Derivation

The original subtree recursively includes only a and its children, in order a,d,f,e. Binary visit(a) first follows first[a]=d through d,f,e, then sibling[a]=b and sib[b]=c, producing a,d,f,e,b,c.

Result and check

b and c are a's siblings, with original parent r, so they are outside a's original subtree. To visit that subtree alone, enumerate children starting at first[a] and recurse on each; do not also follow a's own sibling.

10-E06 Equal head and tail, two states

Derivation

size=0 is empty: enqueue may write index 1, but dequeue must fail. size=4 is full: enqueue must fail, but dequeue may read index 1. Both satisfy tail=(head+size) mod 4. For the full state [E,B,C,D], dequeue at head=1 returns B, then sets head=2 and size=3.

Result and check

The reserved-slot version stores only head/tail, interprets head=tail as empty, and is full when (tail+1) mod 4=head. It holds at most 3 elements. Its empty-slot test cannot be combined with the size-based version's four-element capacity. Check a failing operation before any write so the state remains unchanged.

Elementary data structures solutions (continued)

Verification code appendix: elementary_checks.py

```
"""Exercise actual boundary operations and compare observable sequences to independent
models."""
from collections import deque
from itertools import product
import random


class Ring:
    def __init__(self,capacity):
        self.data=[None]*capacity
        self.head=0
        self.size=0
    def put(self,value):
        if self.size==len(self.data):
            raise OverflowError
        self.data[(self.head+self.size)%len(self.data)]=value
        self.size+=1
    def get(self):
        if self.size==0:
            raise IndexError
        value=self.data[self.head]
        self.head=(self.head+1)%len(self.data)
        self.size-=1
        return value
    def logical(self):
        return [self.data[(self.head+j)%len(self.data)] for j in range(self.size)]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary data structures solutions (continued)

```
class Node:
    def __init__(self,key):
        self.key=key
        self.next=self.prev=None


def insert_between(a,x,b):
    x.prev=a
    x.next=b
    a.next=x
    b.prev=x


def unlink(x):
    a,b=x.prev,x.next
    a.next=b
    b.prev=a


def check_list(sentinel,expected):
    values=[]
    node=sentinel.next
    while node is not sentinel:
        assert node.next.prev is node and node.prev.next is node
        values.append(node.key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary data structures solutions (continued)

```
        assert len(values)<=len(expected)
        node=node.next
    assert values==expected
    assert sentinel.next.prev is sentinel and sentinel.prev.next is sentinel


class Pool:
    def __init__(self,capacity):
        self.next=list(range(1,capacity))+[-1]
        self.prev=[-1]*capacity
        self.used=[False]*capacity
        self.free=0
        self.head=self.tail=-1
    def allocate_before(self,b=-1):
        if self.free==-1:
            raise OverflowError
        x=self.free
        self.free=self.next[x]
        self.used[x]=True
        a=self.tail if b==-1 else self.prev[b]
        self.prev[x]=a
        self.next[x]=b
        if a==-1:
            self.head=x
        else:
            self.next[a]=x
        if b==-1:
            self.tail=x
        else:
            self.prev[b]=x
        return x
    def delete_free(self,x):
        assert self.used[x]
        a,b=self.prev[x],self.next[x]
        if a==-1:
            self.head=b
        else:
            self.next[a]=b
        if b==-1:
            self.tail=a
        else:
            self.prev[b]=a
        self.used[x]=False
        self.next[x]=self.free
        self.free=x
    def validate(self,expected):
        active=[]
        x=self.head
        previous=-1
        while x!=-1:
            assert x not in active and self.used[x] and self.prev[x]==previous
            active.append(x)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elementary data structures solutions (continued)

```
            previous=x
            x=self.next[x]
        assert active==expected and previous==self.tail
        free=[]
        x=self.free
        while x!=-1:
            assert x not in free and not self.used[x]
            free.append(x)
            x=self.next[x]
        assert set(active).isdisjoint(free)
        assert set(active)|set(free)==set(range(len(self.next)))


def main():
    sequences=0
    for length in range(13):
        for operations in product("ED",repeat=length):
            ring=Ring(4)
            reference=deque()
            for step,operation in enumerate(operations):
                before=(ring.data[:],ring.head,ring.size)
                if operation=="E":
                    if len(reference)==4:
                        try:
                            ring.put(step)
                            raise AssertionError("overflow must reject")
                        except OverflowError:
                            assert before==(ring.data,ring.head,ring.size)
                    else:
                        ring.put(step)
                        reference.append(step)
                elif reference:
                    assert ring.get()==reference.popleft()
                else:
                    try:
                        ring.get()
                        raise AssertionError("underflow must reject")
                    except IndexError:
                        assert before==(ring.data,ring.head,ring.size)
                assert ring.logical()==list(reference)
            sequences+=1
    ring=Ring(4)
    ring.put('A');ring.put('B');assert ring.get()=='A'
    for key in "CDE":
        ring.put(key)
    assert ring.data==list("EBCD") and (ring.head,ring.size)==(1,4)
    rng=random.Random(20261024)
    sentinel=Node(None)
    sentinel.next=sentinel.prev=sentinel
    nodes=[]
    for step in range(1000):
        if not nodes or rng.randrange(2):
```

Elementary data structures solutions (continued)

```
            index=rng.randrange(len(nodes)+1)
            b=sentinel if index==len(nodes) else nodes[index]
            x=Node(step)
            insert_between(b.prev,x,b)
            nodes.insert(index,x)
        else:
            x=nodes.pop(rng.randrange(len(nodes)))
            unlink(x)
        check_list(sentinel,[x.key for x in nodes])
    for x in nodes[:]:
        unlink(x)
        nodes.remove(x)
        check_list(sentinel,[x.key for x in nodes])
    pool=Pool(7)
    active=[]
    for _ in range(2000):
        if not active or (len(active)<7 and rng.randrange(2)):
            active.append(pool.allocate_before())
        else:
            x=active.pop(rng.randrange(len(active)))
            pool.delete_free(x)
        pool.validate(active)
    first={'r':'a','a':'d','d':'f'}
    sibling={'a':'b','b':'c','d':'e'}
    def visit(x):
        result=[x]
        child=first.get(x)
        while child is not None:
            result+=visit(child)
            child=sibling.get(child)
        return result
    assert visit('r')==list('radfebc')
    assert visit('a')==list('adfe')
    trace=Pool(5)
    assert [trace.allocate_before() for _ in range(3)]==[0,1,2]
    trace.delete_free(1)
    trace.validate([0,2])
    assert trace.next==[2,3,-1,4,-1] and trace.free==1
    assert trace.allocate_before(2)==1
    trace.validate([0,1,2])
    assert trace.next==[1,2,-1,4,-1] and trace.free==3
    trace.delete_free(2)
    trace.validate([0,1])
    assert trace.next==[1,-1,3,4,-1] and trace.free==2
    print(f"PASS ring_operation_sequences={sequences} overflow_underflow_preserve_state=True
linked_splices=1000 pool_operations=2000 allocation_partition=True
rooted_subtree_boundary=True")


if __name__=="__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary data structures solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study